Food Delivery MongoDB Case Study

Participant Test - Problem Statement, Dataset & Tasks

Food Delivery MongoDB Case Study for Data Engineers

Raw Ingestion, Validation & Incremental Processing of Food Delivery Orders

1 Business Context

A large **food delivery aggregator** operates across multiple Indian cities through:

- Mobile app (Android & iOS)
- Web platform
- Partner restaurant POS integrations

Each customer order generates an **order event** in JSON format. These events are pushed asynchronously from apps, restaurant systems and delivery-partner devices to the central data platform.

The organization faces the following challenges:

- High-volume **semi-structured order data**
- **Schema drift** (new fields such as coupon codes, tips, delivery ratings)
- **Late-arriving orders** due to offline delivery-partner devices syncing later
- **Duplicate or corrected orders** (refunds, partial cancellations, coupon re-application)
- **Invalid records** (missing fields, negative values, unknown order status)
- Need to prepare clean data for **Spark-based analytics**

To address this, the organization uses **MongoDB as a Raw (Bronze) Data Store** in its data engineering architecture.

2 Problem Statement

Design and implement a **MongoDB-based raw ingestion and validation layer** that:

1. Stores food delivery orders without rigid schema constraints
2. Identifies invalid, incomplete or corrupt order records
3. Handles incremental and late-arriving data using an ingestion watermark
4. Resolves duplicate and corrected orders so only the latest valid version is kept
5. Enables aggregation for basic delivery analytics
6. Prepares clean data for downstream Spark / SQL pipelines

3 Dataset Description

Food Delivery Order Event (JSON)

```
{
  "order_id": "ORD5042",
  "customer_id": "CUST881",
  "restaurant_id": "RST07",
  "city": "Pune",
  "cuisine": "North Indian",
  "items_count": 3,
  "order_value": 740,
  "delivery_fee": 40,
  "total_amount": 780,
  "order_status": "DELIVERED",
  "order_time": "2025-02-10T13:05:00Z",
  "delivered_time": "2025-02-10T13:41:00Z",
  "payment_mode": "UPI",
```

```
  "ingested_at": "2025-02-10T13:42:15Z"
}
```

Key Characteristics

- Semi-structured JSON
- Append-heavy workload
- Event time (`order_time`) ≠ arrival time (`ingested_at`)
- Business rule: `total_amount = order_value + delivery_fee (+ tip - discount when present)`
- Valid `order_status` values: PLACED, DELIVERED, CANCELLED
- Occasional:
  - Negative or zero items_count / amount
  - Missing mandatory fields (e.g. city)
  - Unknown order_status values
  - Duplicate order_id
  - Late-arriving records
  - New fields not seen before (coupon_code, discount, tip)

4 Data Engineering Objectives

Participants must demonstrate the ability to:

- Use MongoDB as a **raw ingestion layer**
- Apply **data quality checks** in NoSQL systems
- Separate **valid and rejected records** with a clear rejection reason
- Manage **incremental loads** using arrival timestamps and a watermark
- Handle **late-arriving, duplicate and corrected orders**
- Handle **schema drift** without breaking downstream processing
- Prepare data for **distributed analytics platforms**

5 MongoDB Design (Enterprise Pattern)

Database

`food_delivery_raw`

Collections

Collection	Purpose
`raw_orders`	Raw landing of all order events (never updated or deleted)
`validated_orders`	Cleaned, de-duplicated, latest-version records
`rejected_orders`	Invalid records with a rejection reason
`pipeline_control`	Stores the ingestion watermark and last run details

REAL-TIME DATA FLOW (MULTIPLE BATCHES)

Think of each batch as **15-30 minutes of order activity** during the lunch rush. Insert the batches **one at a time**, running your incremental pipeline (Part D) after each batch.

Batch 1 - Lunch Opening Orders (12:00-12:30)

```
db.raw_orders.insertMany([
  {
    order_id: "ORD3001",
    customer_id: "CUST101",
    restaurant_id: "RST01",
    city: "Bangalore",
    cuisine: "South Indian",
    items_count: 2,
    order_value: 360,
    delivery_fee: 30,
    total_amount: 390,
    order_status: "DELIVERED",
    order_time: ISODate("2025-02-10T11:35:00Z"),
    delivered_time: ISODate("2025-02-10T12:05:00Z"),
    payment_mode: "UPI",
    ingested_at: ISODate("2025-02-10T12:05:40Z")
  },
  {
    order_id: "ORD3002",
    customer_id: "CUST102",
    restaurant_id: "RST02",
    city: "Mumbai",
    cuisine: "Chinese",
    items_count: 3,
    order_value: 540,
    delivery_fee: 40,
    total_amount: 580,
    order_status: "DELIVERED",
    order_time: ISODate("2025-02-10T11:50:00Z"),
    delivered_time: ISODate("2025-02-10T12:18:00Z"),
    payment_mode: "CARD",
    ingested_at: ISODate("2025-02-10T12:19:10Z")
  },
  {
    order_id: "ORD3003",
    customer_id: "CUST103",
    restaurant_id: "RST03",
    city: "Delhi",
    cuisine: "North Indian",
    items_count: 4,
    order_value: 920,
    delivery_fee: 0,
    total_amount: 920,
    order_status: "PLACED",
    order_time: ISODate("2025-02-10T12:25:00Z"),
    payment_mode: "COD",
    ingested_at: ISODate("2025-02-10T12:25:30Z")
  }
])
```

Batch 2 - Peak Lunch Activity (12:30-13:00)

```
db.raw_orders.insertMany([
  {
    order_id: "ORD3004",
    customer_id: "CUST104",
    restaurant_id: "RST04",
    city: "Hyderabad",
    cuisine: "Biryani",
    items_count: 2,
    order_value: 640,
    delivery_fee: 30,
    total_amount: 670,
    order_status: "DELIVERED",
    order_time: ISODate("2025-02-10T12:05:00Z"),
    delivered_time: ISODate("2025-02-10T12:40:00Z"),
    payment_mode: "UPI",
```

```
    ingested_at: ISODate("2025-02-10T12:40:50Z")
  },
  {
    order_id: "ORD3005",
    customer_id: "CUST105",
    restaurant_id: "RST05",
    city: "Chennai",
    cuisine: "South Indian",
    items_count: -2,              // INVALID
    order_value: -300,           // INVALID
    delivery_fee: 30,
    total_amount: -270,          // INVALID
    order_status: "DELIVERED",
    order_time: ISODate("2025-02-10T12:10:00Z"),
    delivered_time: ISODate("2025-02-10T12:43:00Z"),
    payment_mode: "CARD",
    ingested_at: ISODate("2025-02-10T12:44:00Z")
  },
  {
    order_id: "ORD3006",
    customer_id: "CUST106",
    restaurant_id: "RST06",
    // city field missing        // INVALID
    cuisine: "Pizza",
    items_count: 1,
    order_value: 450,
    delivery_fee: 40,
    total_amount: 490,
    order_status: "DELIVERED",
    order_time: ISODate("2025-02-10T12:15:00Z"),
    delivered_time: ISODate("2025-02-10T12:47:00Z"),
    payment_mode: "CARD",
    ingested_at: ISODate("2025-02-10T12:48:00Z")
  },
  {
    order_id: "ORD3007",
    customer_id: "CUST107",
    restaurant_id: "RST07",
    city: "Pune",
    cuisine: "Desserts",
    items_count: 2,
    order_value: 280,
    delivery_fee: 30,
    total_amount: 350,           // INVALID - does not match order_value + delivery_fee
    order_status: "DELIVERED",
    order_time: ISODate("2025-02-10T12:20:00Z"),
    delivered_time: ISODate("2025-02-10T12:51:00Z"),
    payment_mode: "UPI",
    ingested_at: ISODate("2025-02-10T12:52:00Z")
  },
  {
    order_id: "ORD3008",
    customer_id: "CUST108",
    restaurant_id: "RST08",
    city: "Kolkata",
    cuisine: "Bengali",
    items_count: 3,
    order_value: 510,
    delivery_fee: 40,
    total_amount: 550,
    order_status: "DLVRD",        // INVALID - unknown status
    order_time: ISODate("2025-02-10T12:22:00Z"),
    delivered_time: ISODate("2025-02-10T12:54:00Z"),
    payment_mode: "UPI",
    ingested_at: ISODate("2025-02-10T12:55:00Z")
  }
])
```

Batch 3 - Late-Arriving Sync & Schema Drift (13:00-13:30)

```
db.raw_orders.insertMany([
```

```
  {
    order_id: "ORD2995",
    customer_id: "CUST095",
    restaurant_id: "RST01",
    city: "Bangalore",
    cuisine: "North Indian",
    items_count: 1,
    order_value: 220,
    delivery_fee: 30,
    total_amount: 250,
    order_status: "DELIVERED",
    order_time: ISODate("2025-02-10T10:50:00Z"),    // OLD event time
    delivered_time: ISODate("2025-02-10T11:20:00Z"),
    payment_mode: "UPI",
    ingested_at: ISODate("2025-02-10T13:10:00Z")    // Arrived late (offline device)
  },
  {
    order_id: "ORD3009",
    customer_id: "CUST109",
    restaurant_id: "RST09",
    city: "Mumbai",
    cuisine: "Italian",
    items_count: 2,
    order_value: 800,
    delivery_fee: 40,
    coupon_code: "SAVE100",       // NEW field
    discount: 100,                // NEW field
    tip: 20,                      // NEW field
    total_amount: 760,
    order_status: "DELIVERED",
    order_time: ISODate("2025-02-10T12:40:00Z"),
    delivered_time: ISODate("2025-02-10T13:12:00Z"),
    payment_mode: "CARD",
    ingested_at: ISODate("2025-02-10T13:13:00Z")
  }
])
```

Batch 4 - Duplicate, Cancellation & Correction Events (13:30-14:00)

```
db.raw_orders.insertMany([
  {
    order_id: "ORD3002",          // Duplicate ID (exact re-send, no change)
    customer_id: "CUST102",
    restaurant_id: "RST02",
    city: "Mumbai",
    cuisine: "Chinese",
    items_count: 3,
    order_value: 540,
    delivery_fee: 40,
    total_amount: 580,
    order_status: "DELIVERED",
    order_time: ISODate("2025-02-10T11:50:00Z"),
    delivered_time: ISODate("2025-02-10T12:18:00Z"),
    payment_mode: "CARD",
    ingested_at: ISODate("2025-02-10T13:35:00Z")
  },
  {
    order_id: "ORD3003",          // Duplicate ID (status update)
    customer_id: "CUST103",
    restaurant_id: "RST03",
    city: "Delhi",
    cuisine: "North Indian",
    items_count: 4,
    order_value: 920,
    delivery_fee: 0,
    total_amount: 920,
    order_status: "CANCELLED",    // Cancelled by restaurant
    cancellation_reason: "Restaurant closed",
    order_time: ISODate("2025-02-10T12:25:00Z"),
    payment_mode: "COD",
    ingested_at: ISODate("2025-02-10T13:40:00Z"),
```

```
    correction_flag: true
  },
  {
    order_id: "ORD3004",          // Duplicate ID (partial refund)
    customer_id: "CUST104",
    restaurant_id: "RST04",
    city: "Hyderabad",
    cuisine: "Biryani",
    items_count: 1,               // Corrected (one item missing)
    order_value: 320,             // Corrected
    delivery_fee: 30,
    total_amount: 350,            // Corrected
    order_status: "DELIVERED",
    order_time: ISODate("2025-02-10T12:05:00Z"),
    delivered_time: ISODate("2025-02-10T12:40:00Z"),
    payment_mode: "UPI",
    ingested_at: ISODate("2025-02-10T13:50:00Z"),
    correction_flag: true
  }
])
```

6 Tasks for Participants

Complete the parts below in order. Write all queries in **mongosh**. Save every query in a single script file (`solution.js`) with a comment above each step. **Solutions are not provided.**

Part A - Environment Setup

1. Create the database `food_delivery_raw`.
2. Create the four collections listed in Section 5.
3. Insert one control document into `pipeline_control` with:
   - a pipeline name (e.g. orders_bronze_to_silver)
   - an initial watermark of 2025-02-10T00:00:00Z
   - a last-run timestamp and a records-processed counter
5. Create suitable indexes on `raw_orders` to support the incremental load and duplicate lookups. Justify each index in a comment.
6. Create a unique index on `order_id` in `validated_orders`.
7. Insert **Batch 1** only.

Part B - Raw Data Exploration

1. Count the total documents in `raw_orders`.
2. List the distinct cities, cuisines and payment modes present.
3. Find all orders where the gap between order_time and ingested_at is more than 30 minutes.
4. Find documents that contain fields not present in the base schema (Section 3). Write a query that works without hard-coding the new field names.
5. Count documents by order_status.

Part C - Data Quality Rules

Define and implement the following validation rules as MongoDB queries or aggregation expressions:

Rule	Validation rule	Rejection reason
DQ1	Mandatory fields present: order_id, restaurant_id, city, order_time, total_amount, order_status	MISSING_FIELD
DQ2	items_count > 0	INVALID_QUANTITY
DQ3	order_value and total_amount ≥ 0	NEGATIVE_AMOUNT
DQ4	order_status is one of PLACED, DELIVERED, CANCELLED	INVALID_STATUS
DQ5	total_amount = order_value + delivery_fee + tip - discount (missing tip / discount treated as 0)	AMOUNT_MISMATCH
DQ6	order_time ≤ ingested_at, and delivered_time (if present) ≥ order_time	INVALID_TIMESTAMP

1. Write a separate find / aggregate query for each rule that returns the records violating it.
2. Build a single aggregation pipeline that tags each record with an `is_valid` flag and an array `rejection_reasons` (a record can fail more than one rule).
3. Produce a count of records per rejection reason.

Part D - Incremental Load Pipeline

Build a repeatable pipeline that you run **after every batch**:

1. Read the current watermark from `pipeline_control`.

2. Select only records from raw_orders with ingested_at greater than the watermark.
3. Apply the Part C validation logic to the selected records.
4. Write valid records to validated_orders using an **upsert / $merge** keyed on order_id.
5. Write invalid records to rejected_orders with the original record, rejection_reasons and a rejected_at timestamp.
6. Update the watermark to the **maximum ingested_at** of the records processed in this run, and update the run statistics.
7. Insert Batch 2 and re-run steps 1-6. Repeat for Batch 3 and Batch 4.
8. After each run, record in a comment: records read, records validated, records rejected, new watermark.

Part E - Late-Arriving Data

1. Identify the late-arriving record(s) from Batch 3 using a query.
2. Show (with a query) that the record would have been **missed** if the watermark had been based on order_time instead of ingested_at.
3. In 3-4 lines, explain which timestamp should drive incremental loads and which should drive business reporting, and why.

Part F - Duplicates & Corrections

1. Write an aggregation that lists every order_id appearing more than once in raw_orders, with its count and all its ingested_at values.
2. Classify each duplicate as an **exact re-send** or a **correction / status update**.
3. Ensure your Part D pipeline applies the rule **latest ingested_at wins**. An exact re-send must not create an extra document.
4. Add to each record in validated_orders: `version_count` (raw versions seen) and `last_updated_at`.
5. Verify that validated_orders has exactly one document per order_id.
6. Confirm that raw_orders still holds **all** original versions (no updates or deletes on the raw layer).

Part G - Delivery Analytics (on validated_orders)

1. Total revenue (total_amount) by city, highest first - **exclude** cancelled orders.
2. Order count by order_status.
3. Average order value by cuisine.
4. Payment mode share (% of orders per payment mode).
5. Order count per hour, based on order_time.
6. Average delivery time in minutes (delivered_time - order_time) per city, for delivered orders only.
7. Number of orders that used a coupon and the total discount given.

Part H - Prepare Data for Spark / SQL

1. Create a collection `orders_spark_ready` from validated_orders using $out or $merge.
2. Give every document the **same flat schema**: add missing drift fields (coupon_code, discount, tip, cancellation_reason) with default values.
3. Add derived columns: order_date, order_hour, delivery_minutes.
4. Remove MongoDB-internal and pipeline-only fields that analysts do not need.
5. Export orders_spark_ready to CSV and JSON using mongoexport.
6. In 3-4 lines, explain how a Spark job would read this data incrementally.

7 Submission & Evaluation

Submit

- `solution.js` with all queries, commented by Part and step number
- Screenshots or console output of the final counts in raw_orders, validated_orders, rejected_orders and pipeline_control
- The exported CSV / JSON files from Part H
- Written answers for Part E step 3 and Part H step 6

Self-check questions (answer these in your submission)

1. How many records are in raw_orders after all four batches? Why is this more than the number of unique orders?
2. Which orders were rejected, and for which reason(s)?
3. What is the final watermark value, and which record set it?
4. What are the final status and amount of ORD3003 and ORD3004 in validated_orders?
5. Would re-running the pipeline with no new batch change any collection? Why or why not?

Evaluation Rubric

Part	Area	Marks
A	Setup, control document, indexes	10
B	Raw data exploration	10
C	Data quality rules	20
D	Incremental load with watermark	20
E	Late-arriving data	10
F	Duplicates & corrections	15
G	Delivery analytics	10
H	Spark / SQL readiness & export	5
	Total	**100**